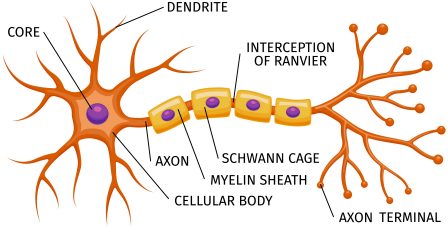
Perceptron

Procheta Sen



Bio Inspired Model

- Neural networks are a model of simulation of the human nervous system.
- Nervous system is composed of nerve cells (neurons).
- Neurons are connected to one another at contact points (synapses).
- Learning is done by changing the strength of synaptic connections between neurons.
- The strength of the connections change in response to external stimuli.
- Perceptron is a model of a single neuron.

Perceptron

- The computation function at a neuron is defined by the weights
- The weights correspond to the strengths of synaptic connections
- The computation function is learned by appropriately changing the weights
- The "external stimulus" is provided by the training data
- Idea: incrementally modify the weights whenever incorrect predictions are made by the current set of weights

Perceptron Diagram

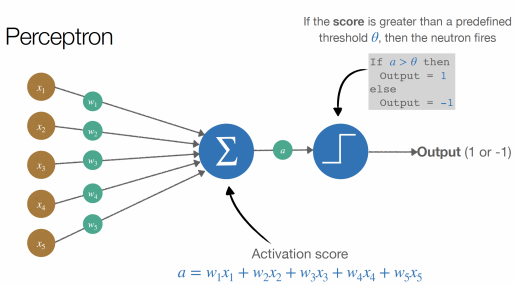


Figure: Diagram of Perceptron

Mathematical Notation

- Input object $\overline{X}^T = (x_1, x_2 \ldots x_d)$
- Weights $\overline{W}^T = (w_1, w_2 \ldots w_d)$
- Activation score $a = \sum_{i=1}^{d} w_i x_i = \overline{W}^T \overline{X}$
- Output 1 if $a > \theta$
- Output -1 if $a \leq \theta$

Bias

- It is convenient to make the threshold θ equal to 0.
- This is achieved by introducing a bias term $b = -\theta$.
- Output 1 if $a > 0$ and Output =-1 $a \leq 0$
- Equivalently, output $sign(\overline{W}^T X + b)$

Notational Trick

- By introducing a feature that is always ON (i.e., $X_0 = 1$ for all objects), we can squeeze the bias term into the weight vector by setting (i.e. $w_0 = b$)

$$a = \sum_{i=1}^{d} w_i x_i = \overline{W}^T \overline{X} \tag{1}$$

- This is more "elegant" as we can write the activation as the inner-product between the weight vector and the feature vector. However, we should keep in mind that bias term still appears in the model.

The Training Algorithm

PerceptronTrain(Training data: D, MaxIter)

$w_i = 0$ for all $i = 1, \ldots, d$;
$b = 0$
for iter $= 1$ ... MaxIter **do**
  for all $(\bar{X}, y) \in D$ **do**
    $a = \bar{W}^T \bar{X} + b$
    if $y \cdot a \leq 0$ **then**
      $w_i = w_i + y \cdot x_i$ for all $i = 1, \ldots, d$
      $b = b + y$

Initialize weights and bias

Compute activation score

Update weights and bias

Test Algorithm

PerceptronTest$(b, w_1, w_2, \ldots, w_d, \mathbf{X})$

1. $a = \mathbf{w}^T\mathbf{X} + b$
2. return $\mathrm{sign}(a)$

Important Features of Perceptron

- Online algorithm: processes objects from the training data set one by one (as opposed to batch learning that requires access to the entire data set, e.g. k-NN)
- Error driven: the parameters are updated only when a test object is classified wrongly using the current parameters (weights and bias).

Detecting misclassification (Incorrect Predictions)

PerceptronTrain(Training data: D, MaxIter)

$w_i = 0$ for all $i = 1, \ldots, d$;

$b = 0$

for iter $= 1 \ldots$ MaxIter **do**

  for all $(\bar{X}, y) \in D$ **do**

    $a = \bar{W}^T \bar{X} + b$

    if $y \cdot a \leq 0$ **then**

      $w_i = w_i + y \cdot x_i$ for all $i = 1, \ldots, d$

      $b = b + y$

return $b, w_1, w_2, \ldots, w_d$

Predicted class $(\text{sign}(a))$ is different from the current instance class (y) if and only if

$$y \cdot a \leq 0.$$

Update Rule, Intuitive Explanation

PerceptronTrain(Training data: D**, MaxIter)**

```
w_i = 0 for all i = 1, ..., d;
b = 0
for iter = 1 ... MaxIter do
    for all (X̄, y) ∈ D do
        a = W̄^T X̄ + b
        if y · a ≤ 0 then
            w_i = w_i + y · x_i, for all i = 1, ..., d
            b = b + y
return b, w_1, w_2, ..., w_d
```

- If we incorrectly classify a **positive** instance as **negative**
  - We should have a **higher activation** to avoid this
  - We **increase** $\overline{W}^T\overline{X}$ and b by **adding** the current instance to the weight vector and the bias
- If we incorrectly classify a **negative** instance as **positive**
  - We should have a **lower activation** to avoid this
  - We **decrease** $\overline{W}^T\overline{X}$ and b by **deducting** the current instance from the weight vector and the bias.

Update Rule, Math Explanation

PerceptronTrain(Training data: D**, MaxIter)**

```
w_i = 0 for all i = 1, ..., d;
b = 0
for iter = 1 ... MaxIter do
    for all (X̄, y) ∈ D do
        a = W̄^T X̄ + b
        if y · a ≤ 0 then
            w_i = w_i + y · x_i, for all i = 1, ..., d
            b = b + y
return b, w_1, w_2, ..., w_d
```

- Current parameters: $b, w_1, w_2, \ldots, w_d$
- Suppose for incoming object $(\overline{X}, y)$ $y = +1$ and $a \leq 0$ **(misclassification)**
- New parameters: $b', w_1', w_2', \ldots, w_d'$

$$a' = \sum_{i=1}^{d} w_i' x_i + b'$$

$$= \sum_{i=1}^{d} (w_i + x_i)\, x_i + (b + 1)$$

$$= \sum_{i=1}^{d} w_i x_i + b \; + \; \sum_{i=1}^{d} x_i x_i \; + \; 1$$

$$= a \; + \; \sum_{i=1}^{d} x_i^2 \; + \; 1 \; > \; a$$

Remark: Activation Adjustment

PerceptronTrain(Training data: D, MaxIter)

```
w_i = 0 for all i = 1, ..., d;
b = 0
for iter = 1 ... MaxIter do
    for all (X̄, y) ∈ D do
        a = W̄^T X̄ + b
        if y · a ≤ 0 then
            w_i = w_i + y · x_i, for all i = 1, ..., d
            b = b + y
return b, w_1, w_2, ..., w_d
```

- There is no guarantee that we will correctly classify a misclassified instance in the next round.
- We have simply increased/decreased the activation but this adjustment might not be sufficient. We might need to do more aggressive adjustments.
- There are algorithms that enforce such requirements explicitly such as the Passive Aggressive Classifier (not discussed here).

Remark: Ordering of Training Instances

PerceptronTrain(Training data: D, MaxIter)

```
w_i = 0 for all i = 1, ..., d;
b = 0
for iter = 1 ... MaxIter do
    for all (X̄, y) ∈ D do
        a = W̄^T X̄ + b
        if y · a ≤ 0 then
            w_i = w_i + y · x_i, for all i = 1, ..., d
            b = b + y
return b, w_1, w_2, ..., w_d
```

- The order of the test instances matters
- Showing only all the positives first and all the negatives next is a bad idea
- Ordering training instances randomly within each iteration produces good results in practice

Remark: Hyperparameter and Overfitting

PerceptronTrain(Training data: D**, MaxIter)**

```
w_i = 0 for all i = 1, ..., d;
b = 0
for iter = 1 ... MaxIter do
    for all (X̄, y) ∈ D do
        a = W̄^T X̄ + b
        if y · a ≤ 0 then
            w_i = w_i + y · x_i, for all i = 1, ..., d
            b = b + y
return b, w_1, w_2, ..., w_d
```

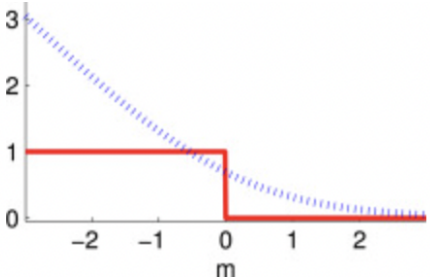
MaxIter is a **hyperparameter** which has to be chosen experimentally.

- If we make many passes over the training data, then the algorithm is likely to **overfit**.
- If we make few passes, this might lead to **underfitting**.

Perceptron

Geometric Interpretation

Procheta Sen



Hyperplane

- The decision in perceptron is made depending on

$$\overline{W}^T\overline{X} + b > 0 \qquad \text{or} \qquad \overline{W}^T\overline{X} + b \leq 0$$

- $\{\overline{X} : \overline{W}^T\overline{X} + b = 0\}$ is the critical region (**decision boundary**)
- $\overline{W}^T\overline{X} + b = 0$ defines a hyperplane

Example:

- In 2D space we have $w_1x_1 + w_2x_2 = 0$ (ignoring the bias term), which is a straight line through the origin.
- In N-dimensional space this is an $(N-1)$-dimensional hyperplane.

Geometric interpretation

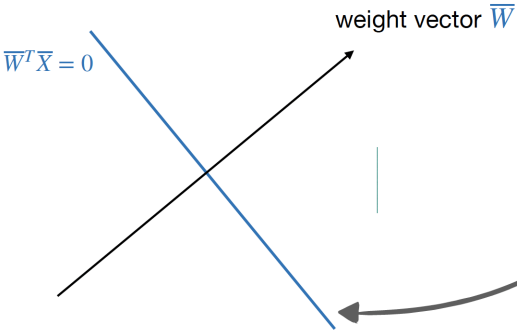


Figure: Hyperplane defined by the weight vector is the line perpendicular to the weight vector.

Geometric Interpretation

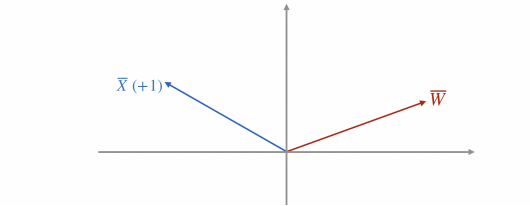


Figure: The angle between the current weight vector $\overline{W}$ and the positive instance $\overline{X}$ is greater than 90°. Therefore, $\overline{W}^T\overline{X} < 0$, and this instance is going to get misclassified as negative.

Geometric Interpretation

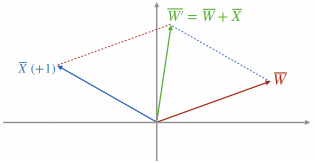


Figure:

- The new weight vector $\overline{W}'$ is the addition of $\overline{W} + \overline{X}$ (as per the perceptron update rule).
- It lies in between $\overline{X}$ and $\overline{W}$.
- Notice that the angle between $\overline{W}'$ and $\overline{X}$ is less than $90°$.
- Therefore, $\overline{X}$ will be classified as positive by $\overline{W}'$.

Vector Algebra Revision

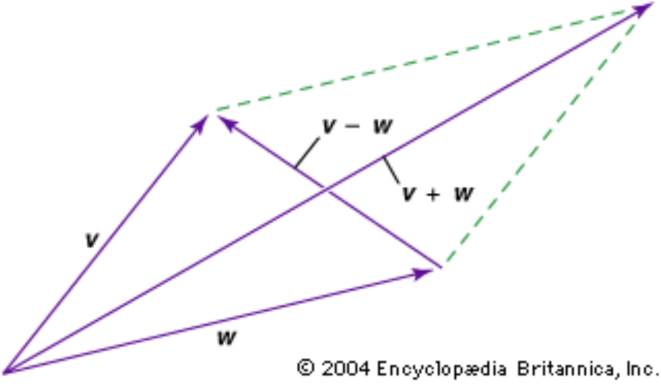


Figure: Vector Algebra Addition

Linear separability

If a given set of positive and negative training instances can be separated into those two groups using a straight line (hyperplane), then we say that the dataset is linearly separable.

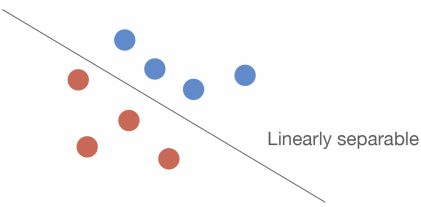


Figure: Vector Algebra Addition

Linear Separability

- When a dataset is linearly separable, there can exist more than one hyperplanes that separates the dataset into positive/negative groups.
- In other words, the hyperplane that linearly separates a linearly separable dataset might not be unique.
- However, (by definition) if a dataset is non-linearly separable, then there exist NO hyperplane that separates the dataset into positive/negative groups.

A non-linearly separable case

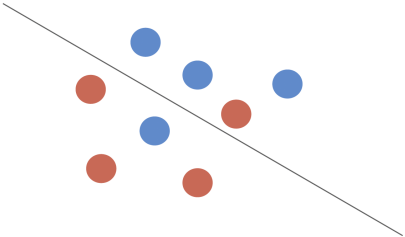


Figure: No matter how we draw straight lines, we cannot separate the red instances from the blue instances

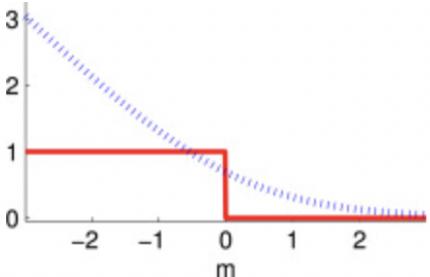
Further Remarks

- When a dataset is linearly separable it can be proved that the perceptron will always find a separating hyperplane!
- The final weight vector returned by the Perceptron is more influenced by the final training instances it sees.
- Take the average over all weight vectors during the training (Averaged Perceptron algorithm)

Perceptron

Loss Function Minimisation

Procheta Sen



Loss Function: Step Function

Let $\mathcal{D} = \{(\overline{X}_1, y_1), \ldots, (\overline{X}_n, y_n)\}$ be the training dataset, where

$$\overline{X}_k = (x_k^{(1)}, x_k^{(2)}, \ldots, x_k^{(d)})^T \quad \text{for every } k = 1, \ldots, n.$$

Let

$$a_k = b + \sum_{i=1}^{d} w_i x_k^{(i)}.$$

Define **loss function** on a single training object $(\overline{X}_k, y_k)$ as

$$L(b, \overline{W}, \overline{X}_k, y_k) = 1 \text{ if } \overline{X}_k \text{ is misclassified, and } L(b, \overline{W}, \overline{X}_k, y_k) = 0, \text{ otherwise.}$$

Define **loss function** for the training dataset $\mathcal{D}$ as

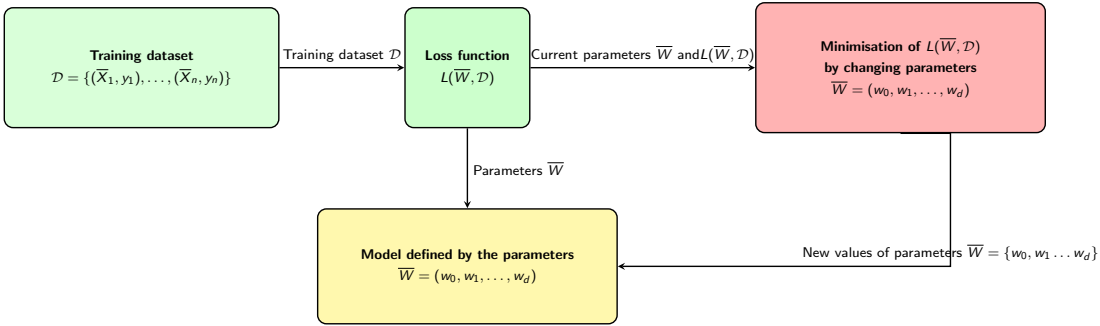
$$L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^{n} L(b, \overline{W}, \overline{X}_k, y_k) = \text{no. of misclassifications.}$$

– this function is piecewise-constant with many discontinuities;
– the derivative of this function (when it exists) is equal to 0;
Hence the gradient descent is not applicable!

Loss Function: Number of Misclassifications

Loss function for the training dataset $\mathcal{D}$ as

$$L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^{n} L(b, \overline{W}, \overline{X}_k, y_k) = \text{no. of misclassifications.}$$



Loss Function $h(t) = \max(0, t)$

Let $\mathcal{D} = \{(\overline{X}_1, y_1), \ldots, (\overline{X}_n, y_n)\}$ be the training dataset, where

$$\overline{X}_k = (x_k^{(1)}, x_k^{(2)}, \ldots, x_k^{(d)})^T \quad \text{for every } k = 1, \ldots, n.$$

Let

$$a_k = b + \sum_{i=1}^{d} w_i \, x_k^{(i)}. \qquad\qquad h(t) = \max(0, t).$$

Define **loss function** for a single object $\overline{X}_k$ as

$$L(b, \overline{W}, \overline{X}_k, y_k) = h(-y_k \cdot a_k).$$

Define **loss function** for the training dataset $\mathcal{D}$ as

$$L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^{n} L(b, \overline{W}, \overline{X}_k, y_k) = \sum_{k=1}^{n} h(-y_k \cdot a_k).$$

Note that

$$L(b, \overline{W}, \overline{X}_k, y_k) = 0, \quad \text{if } \overline{X}_k \text{ is classified correctly;}$$

$$L(b, \overline{W}, \overline{X}_k, y_k) = -y_k \cdot a_k \geq 0, \quad \text{if } \overline{X}_k \text{ is misclassified.}$$

Hence the more misclassifications the model (with parameters b and $\overline{W}$) does, the larger the **loss function** for the training dataset $L(b, \overline{W}, \mathcal{D})$ becomes.

We want to find the parameters b and $\overline{W}$ that minimise $L(b, \overline{W}, \mathcal{D})$!

Loss Function Minimisation

$$L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^{n} L(b, \overline{W}, \overline{X}_k, y_k) = \sum_{k=1}^{n} h(-y_k \cdot a_k)$$

Use the gradient descent method:

$$(b, w_1, \ldots, w_d)^T \leftarrow (b, w_1, \ldots, w_d)^T - \mu \nabla_{b,w_1,\ldots,w_d} L(b, \overline{W}, \mathcal{D})$$

$$\nabla_{b,w_1,\ldots,w_d} L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^{n} \nabla_{b,w_1,\ldots,w_d} L(b, \overline{W}, \overline{X}_k, y_k) = \sum_{k=1}^{n} \nabla_{b,w_1,\ldots,w_d} h(-y_k \cdot a_k)$$

Computation of $\nabla_{b,w_1,\ldots,w_d} h(-y_k \cdot a_k)$

$$h'(t) = 0, \text{ when } t < 0$$

$$h'(t) = 1, \text{ when } t \geq 0 \text{ (h is not differentiable at } t = 0, \text{ but we can extend } h' \text{ by setting } h'(0) = 1)$$

$$\frac{\partial h(-y_k \cdot a_k)}{\partial b} = h'(-y_k \cdot a_k) \cdot \frac{\partial}{\partial b}(-y_k \cdot a_k) = -y_k$$

$$\frac{\partial h(-y_k \cdot a_k)}{\partial w_i} = h'(-y_k \cdot a_k) \cdot \frac{\partial}{\partial w_i}(-y_k \cdot a_k) = -y_k \cdot x_k^{(i)}$$

If $\overline{X}_k$ misclassified, then

$$\nabla_{b,w_1,\ldots,w_d} h(-y_k \cdot a_k) = \left(\frac{\partial h(-y_k \cdot a_k)}{\partial b}, \frac{\partial h(-y_k \cdot a_k)}{\partial w_1}, \ldots, \frac{\partial h(-y_k \cdot a_k)}{\partial w_d} \right)^T$$

$$= -y_k \cdot (1, x_k^{(1)}, x_k^{(2)}, \ldots, x_k^{(d)})^T$$

Otherwise

$$\nabla_{b,w_1,\ldots,w_d} h(-y_k \cdot a_k) = (0, 0, 0, \ldots, 0)^T$$

Loss Function Minimisation

$$\nabla_{b,w_1,\ldots,w_d} L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^{n} \nabla_{b,w_1,\ldots,w_d} L(b, \overline{W}, \overline{X}_k, y_k) = \sum_{k=1}^{n} \nabla_{b,w_1,\ldots,w_d} h(-y_k \cdot a_k)$$

Use the gradient descent method:

$$(b, w_1, \ldots, w_d)^T \leftarrow (b, w_1, \ldots, w_d)^T + \mu \sum_{k:\, \overline{X}_k \text{ misc.}} y_k \cdot \left(1, x_k^{(1)}, x_k^{(2)}, \ldots, x_k^{(d)}\right)^T$$

Batch Gradient Descent

– to make a single update of the parameters we need to use **whole training dataset**

– extremely slow if we have a huge dataset

Online Gradient Descent

Idea: make parameter updates after each misclassification

Instead of

$$(b, w_1, \ldots, w_d)^T \leftarrow (b, w_1, \ldots, w_d)^T - \mu \nabla_{b,w_1,\ldots,w_d} L(b, \overline{W}, \mathcal{D})$$

For a misclassified object $(\overline{X}_k, y_k)$ the update becomes

$$(b, w_1, \ldots, w_d)^T \leftarrow (b, w_1, \ldots, w_d)^T - \mu \nabla_{b,w_1,\ldots,w_d} L(b, \overline{W}, \overline{X}_k, y_k)$$

$$(b, w_1, \ldots, w_d)^T \leftarrow (b, w_1, \ldots, w_d)^T - \mu \nabla_{b,w_1,\ldots,w_d} h(-y_k \cdot a_k)$$

Update Rule for Perceptron

For a misclassified training object $(\overline{X}, y)$ with the activation score

$$a = b + \sum_{i=1}^{d} w_i x_i$$

the weights are updated as follows

$$(b, w_1, \ldots, w_d)^T \leftarrow (b, w_1, \ldots, w_d)^T - \mu \nabla_{b, w_1, \ldots, w_d} h(-y \cdot a)$$

$$\nabla_{b, w_1, \ldots, w_d} h(-y \cdot a) = -y \cdot (1, x_1, x_2, \ldots, x_d)^T$$

$$(b, w_1, \ldots, w_d)^T \leftarrow (b, w_1, \ldots, w_d)^T + \mu\, y \cdot (1, x_1, x_2, \ldots, x_d)^T$$

$$b \leftarrow b + \mu \cdot y$$

$$w_i \leftarrow w_i + \mu \cdot y \cdot x_i \quad \text{for all } i = 1, \ldots, d$$

By setting $\mu = 1$ we obtain exactly the update rule for Perceptron.

The Training Algorithm

PerceptronTrain(Training data: D, MaxIter)

$w_i = 0$ for all $i = 1, \ldots, d$;
$b = 0$
for iter $= 1 \ldots$ MaxIter **do**
  for all $(\overline{X}, y) \in D$ **do**
    $a = \overline{W}^T \overline{X} + b$
    if $y \cdot a \leq 0$ **then**
      $w_i = w_i + y \cdot x_i$, for all $i = 1, \ldots, d$
      $b = b + y$
return $b, w_1, w_2, \ldots, w_d$

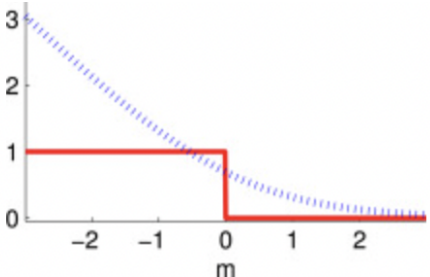
For $\mu = 1$

$w_i \leftarrow w_i + y \cdot x_i$

$b \leftarrow b + y$

Classifier Evaluation

Procheta Sen



Evaluation of a Classifier/Model/System

- How **good** is it?
- Levels of **goodness**
  - **Absolute goodness:** When we run our trained model in the "wild" it does what we expect it to do
  - no way of knowing *before* we deploy the model and when we do know, too late!
- **Relative goodness:** We have a small representative sample of **test data**
  - We compare the output produced by our classifier on this test dataset (*gold standard*) and measure how well it resembles the labels in the dataset

Gold Standard

- A dataset that we use for evaluation purpose. Also known as **test data**.
- Each test instance in the test data has its correct label annotated.
- Numerous measures exist (as we will shortly see) to **compare** the **predicted** labels by the trained classifier and actual (**target**) labels in the test dataset.
- **Never train on test data!!!**

Confusion Matrix (error matrix)

		Actual	
		Actual YES(+)	Actual NO(-)
Predicted	Predicted YES(+)	True Positives (TP)	False Positives (FP)
	Predicted NO(-)	False Negatives (FN)	True Negatives (TN)

The matrix makes it easy to see if the system is **confusing** two classes.

Definitions

- **True Positive**

  We predicted as positive and it is indeed positive
- **True Negative**

  We predicted as negative and it is indeed negative
- **False Positive**

  We predicted as positive but it turns out to be negative
- **False Negative**

  We predicted as negative but it turns out to be positive

		Actual	
		Actual YES(+)	**Actual** NO(-)
Predicted	**Predicted** YES(+)	**True Positives** (TP)	**False Positives** (FP)
	Predicted NO(-)	**False Negatives** (FN)	**True Negatives** (TN)

Example

- We have a set of images some of which contain a car
- We want to detect photos with a car
- Positive class (+): all photos with a car
- Negative class (-): all other photos

Example: Detecting Cancer

- Let assume we trained a classifier to detect cancer based on some features.
- Predicting YES means we predict that the patient has cancer.
- We predicted the patient as having cancer but further tests revealed that the patient does not have cancer
  - False Positive
- We predicted the patient as not having cancer (so no further tests were done) but the patient died with cancer!
  - False Negative
- The moral of the story
  - FP and FN have very different importance in real-world data mining tasks.

Evaluation Measures

- Accuracy
- Precision
- Recall
- F-score
- Many more (see e.g. Confusion matrix)

		Actual	
		Actual YES(+)	**Actual** NO(-)
Predicted	**Predicted** YES(+)	**True Positives** (TP)	**False Positives** (FP)
	Predicted NO(-)	**False Negatives** (FN)	**True Negatives** (TN)

Evaluation Measures: Accuracy

Answers to

what proportion of all objects were **correctly classified**?

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

		Actual	
		Actual YES(+)	**Actual** NO(-)
Predicted	**Predicted** YES(+)	**True Positives** (TP)	**False Positives** (FP)
	Predicted NO(-)	**False Negatives** (FN)	**True Negatives** (TN)

Evaluation Measures: Precision

Answers to

what proportion of **predicted Positives** is truly Positive?

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

		Actual	
		Actual YES(+)	**Actual** NO(-)
Predicted	**Predicted** YES(+)	**True Positives** (TP)	**False Positives** (FP)
	Predicted NO(-)	**False Negatives** (FN)	**True Negatives** (TN)

Evaluation Measures: Recall

Answers to

what proportion of **actual Positives** is correctly classified?

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

		Actual	
		Actual YES(+)	**Actual** NO(-)
Predicted	**Predicted** YES(+)	**True Positives** (TP)	**False Positives** (FP)
	Predicted NO(-)	**False Negatives** (FN)	**True Negatives** (TN)

What is more important: Precision or Recall?

Application 1 (cancer detection)

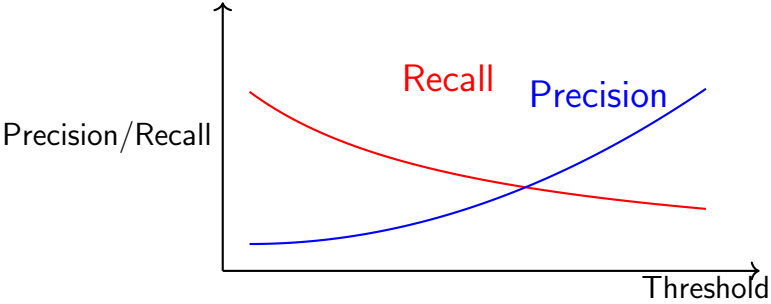
- Positive: has cancer
- Negative: healthy
- We want to detect as many patients with cancer as possible
- We want to have **high recall**

Application 2 (product recommendation)

- Positive: relevant products
- Negative: non-relevant products
- We want to make sure that almost all recommended products are relevant to the user
- We want to have **high precision**

There is a **trade-off** between precision and recall: improving precision often results in lowering recall and vice versa

Precision–Recall Trade-off



By simply varying the **threshold** of our cancer detector we can get a **high precision** OR **low recall** system. There is a **trade-off**.

Evaluation measures: F-score

$$\text{F-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F-score is the **harmonic mean** between precision and recall

$$\text{F-score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}}$$

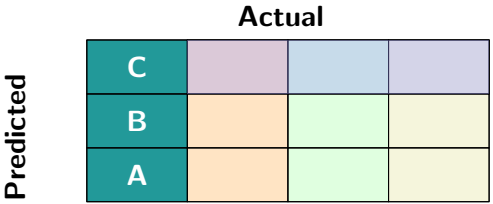
		Actual	
		Actual YES(+)	**Actual** NO(-)
Predicted	**Predicted** YES(+)	**True Positives** (TP)	**False Positives** (FP)
	Predicted NO(-)	**False Negatives** (FN)	**True Negatives** (TN)

- F-score is in between precision and recall
- F-score gives a larger weight to lower numbers

Evaluation Measures for Multiple Classes

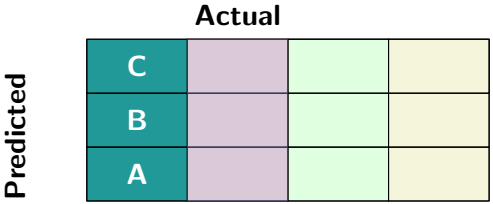
Precision for a class A:

$$\frac{\text{no. objects correctly classified } A}{\text{no. objects classified } A}$$



Recall for class A:

$$\frac{\text{no. objects correctly classified } A}{\text{no. objects that belong to class } A}$$



Evaluation Measures for Multiple Classes

F-score for class A

$$\text{F-score}_A = \frac{2 \times \text{Precision}_A \times \text{Recall}_A}{\text{Precision}_A \; + \; \text{Recall}_A}$$

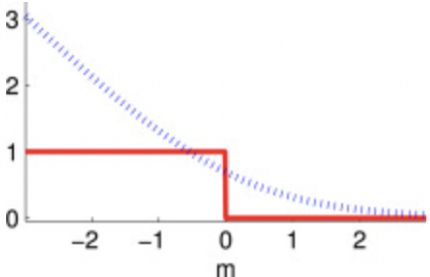
Macro F-score

$$\text{Macro F-score} = \frac{1}{C} \sum_{i=1}^{C} \text{F-score}_i$$

where C is the number of classes and F-score_i is the F-score for class i.

MultiClass Classification

Procheta Sen



Classification algorithms

Multiclass classifiers

- k-NN
- Naive Bayes

Binary classifiers

- Perceptron
- Logistic regression

How to turn a binary classifier to a multiclass classifier?

Given binary classification algorithm $\mathcal{A}$ we want to design a meta-algorithm that uses $\mathcal{A}$ to make k-class predictions.

Two strategies

- **One-vs.-one** approach
- **One-vs.-rest** approach

One-vs.-one approach

1. For each pair of classes i and j, use training objects from classes i and j to train algorithm $\mathcal{A}$ to distinguish between objects in classes i and j. Denote the obtained classifier $A_{i,j}$.
2. This results in
$$\frac{k(k-1)}{2}$$
prediction models.
3. Applying the prediction model $A_{i,j}$ to an incoming object $\overline{X}$ is interpreted as voting. $A_{i,j}$ votes either $+1$ for $\overline{X}$ to be in class i, or $A_{i,j}$ votes $+1$ for $\overline{X}$ to be in class j.
4. For an incoming object $\overline{X}$, apply all prediction models one by one.
5. The class label with the most votes is declared as the winner.

Frame Title

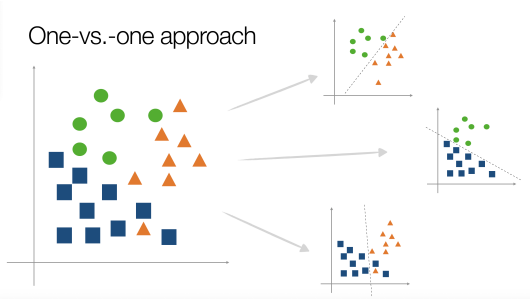


Figure: Caption

One-vs.-one approach: drawbacks

- There might be ambiguity if some classes get the same number of votes (if the binary classifier $\mathcal{A}$ can produce a confidence score, it can be used to break ties).

One-vs.-rest approach

In this approach we assume that the binary classification algorithm $\mathcal{A}$ can output a numeric score representing its "confidence" that an object belongs to a particular class.

1. For each class i, train the binary classifier $\mathcal{A}$ with the objects of class i treated as positive samples and all other objects as negative samples. Denote the obtained classifier A_i.
2. This results in k prediction models.
3. For an incoming object $\overline{X}$, apply all prediction models $A_1, A_2, \ldots, A_k$.
4. Output for object $\overline{X}$ the class label y corresponding to the model with the highest score, i.e.,

$$y = \arg \max_{i \in \{1,2,\ldots,k\}} A_i(\overline{X})$$

One Vs. Rest Approach

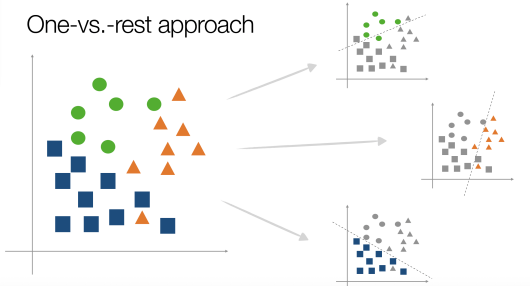


Figure: Caption

One-vs.-rest approach

The choice of the numeric score depends on the classifier at hand.

1. For Perceptron: the activation score

$$a = b + \overline{W}^T \overline{X}$$

2. For Logistic regression: $\sigma(a)$, where

$$a = b + \overline{W}^T \overline{X}$$

One-vs.-rest approach: drawbacks

- the scale of the confidence scores may differ between the binary classifiers
- the binary classifiers are trained on unbalanced datasets: usually the set of negative objects will be much larger than the set of positive objects